

Multi-Tool Research Agent

A Grounded, Tool-Calling LLM Agent with Memory,
Caching, Cost Control, and Evaluation

Technical Architecture & Implementation Report

Project 2 — AI Engineering Roadmap (Agentic AI Foundations)

Stack: Python 3.12 · OpenAI (gpt-4o-mini) · Pydantic · FastAPI · SQLite · Docker

Tools: Web Search · arXiv · Wikipedia · YouTube Transcripts · Calculator

June 2026

Table of Contents

1. Executive Summary

- What the system is
- Headline results

2. Problem Statement & Motivation

- Why grounding matters
- Why tool-calling is the foundation of agentic AI

3. Core Concepts

- Tool calling
- Native tool calling vs. ReAct
- JSON Schema as the tool contract
- The ToolResult envelope

4. System Architecture

- The request lifecycle
- Module map & dependency graph
- Control flow of the agentic loop

5. Component Deep-Dives

- Configuration layer
- Schemas & data contracts
- The five tools
- Tool registry & dispatch
- Conversation memory
- The agent loop & cost control
- Caching layer
- HTTP API

6. Production Engineering

- Graceful degradation
- Hard timeouts & the threading model
- Cost caps
- Caching strategy & staleness
- Security posture

7. Evaluation

- Methodology & design decisions
- Results
- What the eval surfaced

8. Deployment

Containerization

Continuous integration

Hosting

9. Limitations & Future Work

10. Appendix: Glossary & File Index

1. Executive Summary

1.1 What the system is

The Multi-Tool Research Agent is a Large Language Model (LLM) application that answers questions by deciding, per query, which external tool to invoke, executing that tool, reading the result, and synthesizing a final answer grounded in real retrieved data with inline source citations. It is not a single canned API call: the model drives a multi-step loop, choosing one of five tools — general web search, arXiv academic search, Wikipedia lookup, YouTube transcript retrieval, and a sandboxed calculator — or answering directly from its own knowledge when no tool is warranted.

The system is exposed through three interchangeable interfaces backed by a single core: a one-shot command-line invocation, an interactive REPL with persistent conversation memory, and an HTTP API suitable for deployment. The same agent logic serves all three.

1.2 Headline results

Dimension	Result
Tool-selection accuracy	96.0% (24/25) on a labeled evaluation set
Tools integrated	5 (web, arXiv, Wikipedia, YouTube, calculator)
Cost per request	Typically under US\$0.002; hard cap at US\$0.05
Failure handling	Structured errors + automatic fallback to alternate tools
Caching	SQLite, 24-hour TTL, keyed on validated arguments
Memory	Token-budgeted sliding window preserving tool-call integrity
Interfaces	CLI, interactive REPL, HTTP API (FastAPI), Docker container

2. Problem Statement & Motivation

2.1 Why grounding matters

An LLM is a frozen text predictor. Its knowledge ends at a training cutoff, and it has no intrinsic access to the live world: it cannot report today's news, cannot retrieve a paper published last week, and cannot compute an exact arithmetic result reliably. Asked a time-sensitive or computational question, an ungrounded model produces fluent but potentially stale or fabricated output. The defining failure mode observed early in development was precisely this — the model confidently answering a question about current regulation with information frozen at its training cutoff, presented as if current.

Grounding addresses this by routing the model's information needs through external tools whose outputs are current and verifiable, then requiring the model to cite the specific source of each claim. The result is a system that "goes and finds out" rather than "guesses from memory."

2.2 Why tool-calling is the foundation of agentic AI

The single most important capability in this project is tool calling: the mechanism by which an LLM emits a structured request to invoke a function, the application executes it, and the result is fed back into the conversation. The model never executes anything itself; it only *decides* what to do based on the tool descriptions it is given. This decision — choose a tool, observe the result, decide the next step — is the atomic unit of agentic behavior. Every more sophisticated agent (multi-agent systems, planners, reasoning loops) is built on this same primitive. Mastering it is the prerequisite for everything that follows in agentic AI.

3. Core Concepts

3.1 Tool calling

Modern LLMs are post-trained to recognize a list of available tools — each described by a name, a natural-language description, and a JSON Schema for its arguments — and to emit a structured JSON request when a tool is appropriate. The application detects this request, runs the corresponding function, and appends the result to the message history as a dedicated message. The model then continues, now able to read the tool's output.

Crucially, tool selection is statistical, not deductive. The model pattern-matches the user's intent against the tool descriptions and argument names. Vague descriptions produce poor routing; precise, example-laden descriptions produce reliable routing. The tool descriptions and the system prompt are therefore the two primary levers controlling agent behavior.

3.2 Native tool calling vs. ReAct

An earlier paradigm, ReAct, prompted the model to emit reasoning and actions as free text in a strict textual format ("Thought / Action / Observation"), which the application parsed with regular expressions. This is brittle — a single malformed line breaks the parser — and verbose. Native tool calling, used throughout this project, instead relies on the model emitting structured JSON validated by the provider's SDK. It is more reliable, supports parallel tool calls, and eliminates fragile text parsing. ReAct remains relevant only for models without native tool-calling support.

3.3 JSON Schema as the tool contract

Each tool's arguments are defined as a JSON Schema — the formal contract between the model and the application. In this project, those schemas are authored as Pydantic models and converted to the provider's tool-definition format programmatically. This yields a single source of truth: the same Pydantic class both (a) generates the schema the model sees and (b) validates the arguments the model returns before the tool runs. Tight schemas — with enumerated value sets, bounded numeric ranges, and explicit descriptions — reduce hallucinated or malformed arguments.

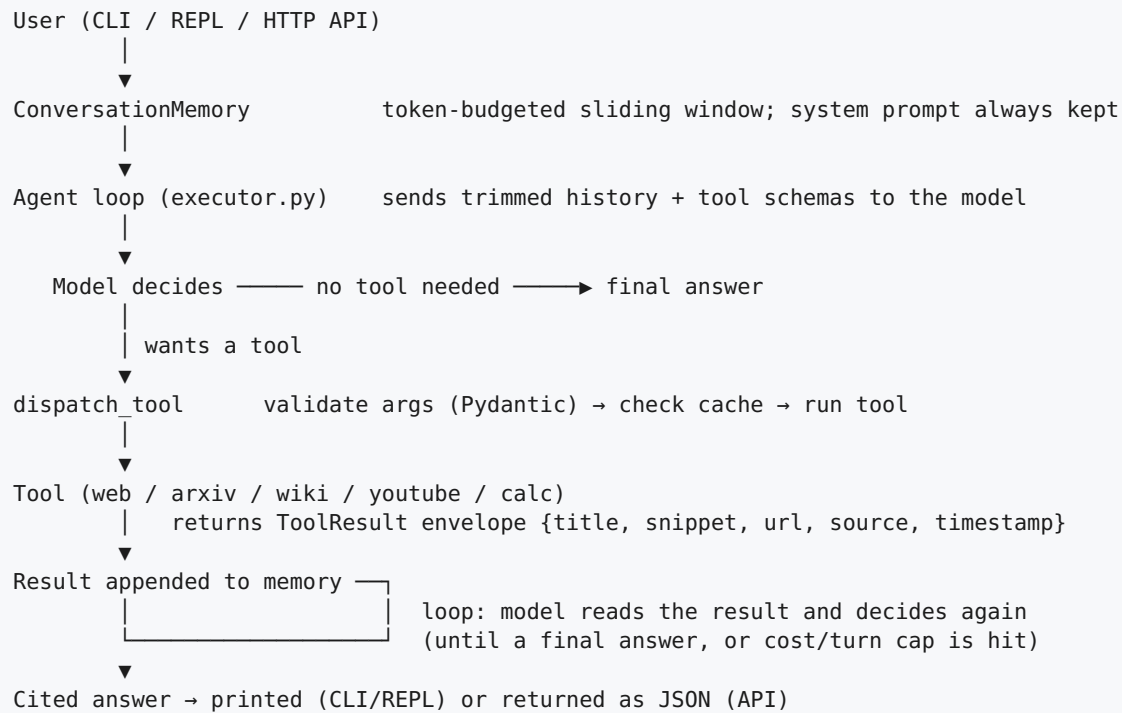
3.4 The ToolResult envelope

Every retrieval tool returns its results in one uniform structure, the `ToolResult` envelope, carrying five fields: `title`, `snippet`, `url`, `source`, and `timestamp`. Normalizing heterogeneous tool outputs (web hits, papers, articles, transcripts) into one shape yields three benefits: citations become trivial (the model is instructed to cite the `url` of each claim), freshness is explicit (the `timestamp` tells the model how current the data is), and the underlying provider becomes swappable without changing the model-facing contract.

4. System Architecture

4.1 The request lifecycle

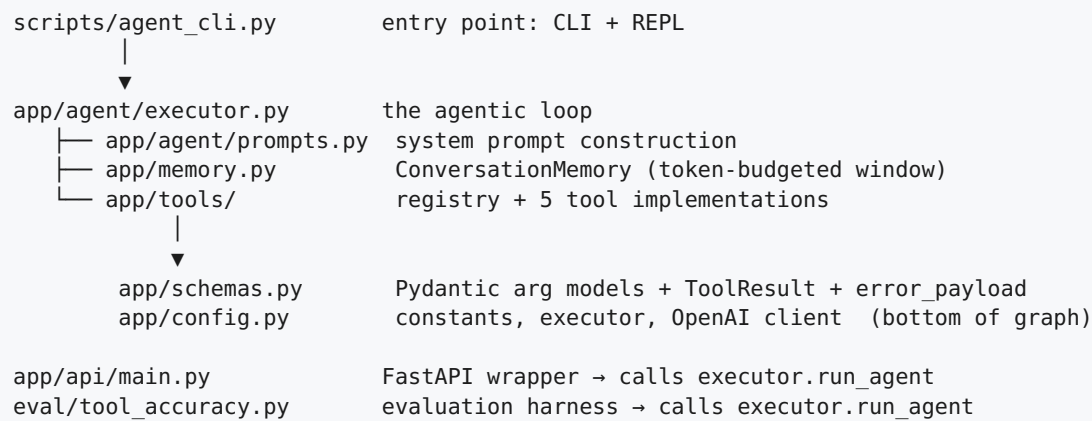
A single question traverses the system as follows. The diagram traces one query from entry to a cited answer, including the loop that distinguishes an agent from a single tool call.



The defining characteristic is the feedback loop: a tool result is never the end of processing. It is appended to the conversation and the model is invoked again, now able to decide whether the answer is complete, whether another tool is needed, or whether to retry. The loop is bounded by a hard turn cap (`MAX_TURNS = 5`) and a per-request cost cap, guaranteeing termination.

4.2 Module map & dependency graph

The codebase is organized as an installable package with a strict, acyclic dependency graph. Lower layers know nothing of higher ones.



File	Responsibility
<code>app/config.py</code>	Shared constants, timeouts, the thread-pool executor, the OpenAI client, environment loading. No internal dependencies.
<code>app/schemas.py</code>	All Pydantic argument models, the ToolResult envelope, and the uniform error payload helper.
<code>app/memory.py</code>	Token counting and the ConversationMemory sliding-window class.
<code>app/tools/*.py</code>	One file per tool (web, arxiv, wiki, youtube, calc) plus the registry.
<code>app/agent/prompts.py</code>	System prompt: date anchor, tool-selection rules, response-shape contract.
<code>app/agent/executor.py</code>	The agentic loop, cost tracking, tool dispatch orchestration.
<code>app/cache.py</code>	SQLite-backed tool-result cache with TTL.
<code>app/api/main.py</code>	FastAPI application exposing <code>/health</code> and <code>/query</code> .
<code>scripts/agent_cli.py</code>	Thin CLI/REPL entry point.
<code>eval/tool_accuracy.py</code>	Tool-selection accuracy evaluation harness.

4.3 Control flow of the agentic loop

For a representative multi-tool query — "Compare Marie Curie's work (Wikipedia) with recent arXiv papers on radioactivity" — the loop executes as follows:

1. The question becomes a `user` message and enters memory.
2. The trimmed message history plus the tool schemas are sent to the model.
3. The model emits a tool call for `search_wikipedia`; the application validates the arguments, checks the cache, runs the tool, and appends the `ToolResult` JSON as a `tool` message.
4. The loop repeats; the model now emits a tool call for `search_arxiv`; the result is appended.

5. The loop repeats; with both results in context, the model produces a final answer with no further tool calls, citing both sources. The loop terminates and the answer is returned.

5. Component Deep-Dives

5.1 Configuration layer

All cross-cutting constants live in one dependency-free module: the model identifier, the loop cap, per-tool timeout values, token cost rates, the cost cap, the shared thread pool, and the single reused OpenAI client. Centralizing these eliminates magic numbers scattered across the codebase and provides one obvious place to tune behavior. Environment variables (notably the API key) are loaded before the client is constructed.

5.2 Schemas & data contracts

Five Pydantic models define the tool inputs, each with bounded and enumerated fields where appropriate — for example, the arXiv category is constrained to a fixed set of valid subject codes, and result counts are bounded to sane ranges. The `ToolResult` envelope defines the uniform output. A shared `error_payload` helper produces the single error JSON shape every tool returns on failure, ensuring the model encounters a consistent, recognizable failure signal.

5.3 The five tools

Tool	Backend	Notes
<code>search_web</code>	DuckDuckGo (ddgs)	General current-events search; recency filter mapped from human labels to provider codes.
<code>search_arxiv</code>	arXiv API	Academic preprints; category filtering and client-side date filtering; abstracts truncated to control context size.
<code>search_wikipedia</code>	wikipedia-api	Encyclopedic facts; descriptive User-Agent required by Wikipedia; long articles capped.
<code>fetch_youtube_transcript</code>	youtube-transcript-api	Accepts URL or bare ID; transcript capped to control context size.
<code>calculator</code>	numexpr	Sandboxed math evaluation — never uses <code>eval()</code> , eliminating the code-execution attack surface.

Each network tool follows an identical resilience pattern: the work is submitted to a shared thread pool and awaited with a hard timeout; on success it returns a JSON array of `ToolResult` objects; on any failure it returns the uniform error payload.

5.4 Tool registry & dispatch

The registry converts each Pydantic model into the provider's tool-definition format, assembles the list of tools sent to the model, and exposes a single `dispatch_tool` entry point. Dispatch performs three steps in order: validate the model's raw arguments against the Pydantic schema; consult the cache; on a miss, run the tool and conditionally store the result. The conditional store is a deliberate safeguard —

error results are never cached, preventing a transient failure from being replayed for the entire cache lifetime.

5.5 Conversation memory

The `ConversationMemory` class maintains the full message history and, on each request, returns a trimmed copy that fits within a configurable token budget. Three invariants are enforced: the system message is never dropped; messages are kept newest-first (recent context is most relevant); and an assistant tool-call message and its corresponding tool-result messages are always trimmed as a single group. The last invariant is mandatory — the provider's API rejects any tool-result message that lacks its preceding tool-call, so partial groups are never produced. When the budget is smaller than a single group, the system degrades to returning only the system message rather than emitting an invalid request.

5.6 The agent loop & cost control

The loop sends the trimmed history to the model, reads any tool calls, executes them via dispatch, appends results, and repeats. After every model call it reads the exact token usage reported by the provider and accumulates the real cost using published per-token rates. Before each iteration it checks the accumulated cost against the cap; if exceeded, it aborts and reports the actual spend. Using the provider's reported usage — rather than a local estimate — makes cost accounting exact.

5.7 Caching layer

A SQLite database stores tool results keyed by a stable hash of the tool name and its *validated* arguments. Caching on validated arguments (after Pydantic fills defaults) normalizes cosmetically different but semantically identical calls into a single cache entry, raising the hit rate. Entries carry a creation timestamp and expire after a 24-hour time-to-live; expired entries are evicted on read. SQLite parameterized queries are used throughout, eliminating SQL-injection risk.

5.8 HTTP API

A FastAPI application wraps the agent with two endpoints: a `/health` liveness probe (used by container and hosting platforms) and a `/query` endpoint that accepts a question and returns the answer together with the list of tools the agent used. The API is stateless — each request is independent — which is the correct default for a horizontally scalable service; session-scoped memory is identified as future work.

6. Production Engineering

6.1 Graceful degradation

Tool failures are treated as data, not exceptions. A failed tool returns a structured error payload that the model is explicitly instructed to handle: it may rephrase and retry once, fall back to an alternate tool, or answer from general knowledge with a clear caveat. This behavior was verified repeatedly during development — for instance, when the Wikipedia backend timed out, the agent automatically fell back to web search and still produced a correctly cited answer.

6.2 Hard timeouts & the threading model

Each network tool's work is submitted to a module-level thread pool and awaited with a bounded timeout, guaranteeing a wall-clock ceiling on every tool call. The pool is deliberately module-level rather than wrapped in a context manager: a context-managed pool blocks on shutdown until its worker completes, which would defeat the timeout. Because threads cannot be forcibly killed in CPython, a timed-out worker is allowed to expire in the background while the application proceeds. A true hard kill of untrusted code would require a subprocess; this is documented as the boundary of the current approach.

6.3 Cost caps

Every request carries a hard monetary ceiling (default US\$0.05). Because cost is accumulated from the provider's exact token usage, the cap is precise rather than estimated. This bounds the blast radius of a runaway loop or an adversarial prompt designed to force repeated expensive calls.

6.4 Caching strategy & staleness

The single global 24-hour TTL is the simplest correct policy. Its known limitation is staleness for volatile queries: a "latest news" result cached for 24 hours may return yesterday's news. The documented next step is differentiated TTLs — short for volatile sources (news), long for stable ones (encyclopedia articles, published papers) — and optionally keying on the recency parameter.

6.5 Security posture

Surface	Mitigation
Calculator code execution	numexpr evaluator only; <code>eval()</code> never used.
SQL injection (cache)	Parameterized queries exclusively.
Secret management	API key loaded from environment / <code>.env</code> ; never committed; excluded from the Docker image.
Untrusted tool arguments	All model-supplied arguments validated against Pydantic schemas before use.
Unbounded cost	Per-request cost cap and turn cap.

7. Evaluation

7.1 Methodology & design decisions

The evaluation measures tool-selection accuracy: for each labeled question, did the agent invoke the expected tool? This metric is objective (no judge model required), cheap, and directly tied to the agent's core responsibility — routing. The harness runs each of 25 questions through the real agent, captures the tools actually called, and scores against the expected tool(s). Three deliberate design decisions govern scoring:

- **"Present among," not exclusivity.** A question is correct if the expected tool is among those called. A legitimate comparison question may call several tools; penalizing that would measure restraint rather than routing.
- **"None" means no research tool fired.** Questions answerable from the model's own knowledge are labeled `none` and are correct only if no tool was called.
- **Acceptable alternatives are allowed.** A question may list multiple acceptable tools (e.g. Wikipedia or web); any one suffices. To keep the metric falsifiable, `none` is forbidden from co-existing with real tools in a label — a rule the harness enforces by rejecting malformed labels.

7.2 Results

TOOL-SELECTION ACCURACY: 24/25 = 96.0%

calculator	5/5 = 100%
search_web	5/5 = 100%
search_arxiv	5/5 = 100%
search_wikipedia	4/4 = 100%
fetch_youtube_transcript	3/3 = 100%
none	1/2 = 50%

7.3 What the eval surfaced

The harness earned its place by finding issues in the system and in its own specification, not merely confirming expectations:

- **A genuine tool bug.** The calculator raised an error when an expression referenced the constant `pi`, because the underlying evaluator does not define mathematical constants. This is a concrete defect the eval caught that ad-hoc testing had missed.
- **A specification ambiguity.** The question "What is the capital of France?" was labeled `none`, but the agent looked it up on Wikipedia. Both behaviors are defensible — parametric recall versus grounded, cited retrieval — which is why that row scores 50%. The value of the eval here is that it forced an explicit decision about whether grounding stable facts is desirable or wasteful, rather than leaving the behavior unexamined.

Principle. An evaluation that only confirms existing beliefs is theater. One that surfaces defects in the implementation and disagreements between the specification and reality is doing the real work — it locates exactly where design, labels, and behavior diverge.

8. Deployment

8.1 Containerization

The application ships as a Docker image built on a slim Python base. Dependency manifests are copied and installed before the application code, exploiting layer caching so that code changes do not trigger dependency reinstalls. Secrets and local cache files are excluded from the image. The container binds to all interfaces so it is reachable when hosted, and exposes the FastAPI service via uvicorn.

8.2 Continuous integration

A GitHub Actions workflow runs on every push and pull request, installing dependencies on a clean machine and verifying that the package imports and compiles. This catches broken imports and syntax errors before they reach a deployment. Running the full evaluation in CI (which requires an API key as a managed secret) is identified as a future enhancement.

8.3 Hosting

The container is deployed to a managed platform that auto-detects the Dockerfile, injects the API key as a secret environment variable, and uses the `/health` endpoint as its liveness probe. On free-tier hosting the service may cold-start after inactivity, introducing a one-time delay on the first request after idle.

9. Limitations & Future Work

Area	Current state	Planned improvement
Streaming	Full answer returned at once	Token streaming to reduce time-to-first-token
HTTP memory	<code>/query</code> is stateless	Session-scoped memory keyed by a session identifier
Rate limiting	None (mitigated by caching)	Per-tool outbound rate limiting
Tool concurrency	Tool calls within a turn run sequentially	Parallel execution of independent calls (single-layer)
Cache policy	Single global 24h TTL	Differentiated TTLs per tool type
Observability	Debug-gated stderr tracing	LangSmith tracing and CI-gated regression eval

10. Appendix: Glossary & File Index

Glossary

Term	Definition
Tool calling	The mechanism by which an LLM emits a structured request to invoke an application-defined function.
Agentic loop	A loop in which the model repeatedly observes tool results and decides the next action until producing a final answer.
ToolResult envelope	The uniform output structure (title, snippet, url, source, timestamp) returned by all retrieval tools.
Token budget	A ceiling on the number of tokens of conversation history sent to the model per request.
Cost cap	A hard monetary ceiling per request, enforced from the provider's reported token usage.
TTL	Time-to-live; the duration a cached entry remains valid before expiry.
Graceful degradation	Continuing to provide useful service when a component (here, a tool) fails.
Tool-selection accuracy	The fraction of evaluation questions for which the agent invoked an expected tool.

File Index

Path	Purpose
<code>app/config.py</code>	Constants, timeouts, thread pool, OpenAI client, env loading.
<code>app/schemas.py</code>	Pydantic argument models, ToolResult envelope, error payload helper.
<code>app/memory.py</code>	Token counter and ConversationMemory sliding window.
<code>app/cache.py</code>	SQLite tool-result cache with 24h TTL.
<code>app/tools/web.py</code>	DuckDuckGo web search tool.
<code>app/tools/arxiv.py</code>	arXiv academic search tool.
<code>app/tools/wiki.py</code>	Wikipedia summary tool.
<code>app/tools/youtube.py</code>	YouTube transcript tool.
<code>app/tools/calc.py</code>	Sandboxed numexpr calculator.
<code>app/tools/registry.py</code>	Schema conversion, TOOLS list, dispatch_tool.
<code>app/agent/prompts.py</code>	System prompt construction.
<code>app/agent/executor.py</code>	The agentic loop and cost control.
<code>app/api/main.py</code>	FastAPI wrapper (/health, /query).
<code>scripts/agent_cli.py</code>	CLI and interactive REPL entry point.
<code>eval/dataset.jsonl</code>	25 labeled evaluation questions.
<code>eval/tool_accuracy.py</code>	Tool-selection accuracy harness.

End of report.